Basam Mozan *Professor Paula Branco*
300275138 *CSI5388[F] AI-For Cybersecurity App*

Assignment 3 Report

- **Static Evaluation:**

First, I read the static data and checked how many records there were. Then I displayed the first few records to check the nature of the data and how many columns are there. After successfully reading and displaying data, I move to the analysis part.

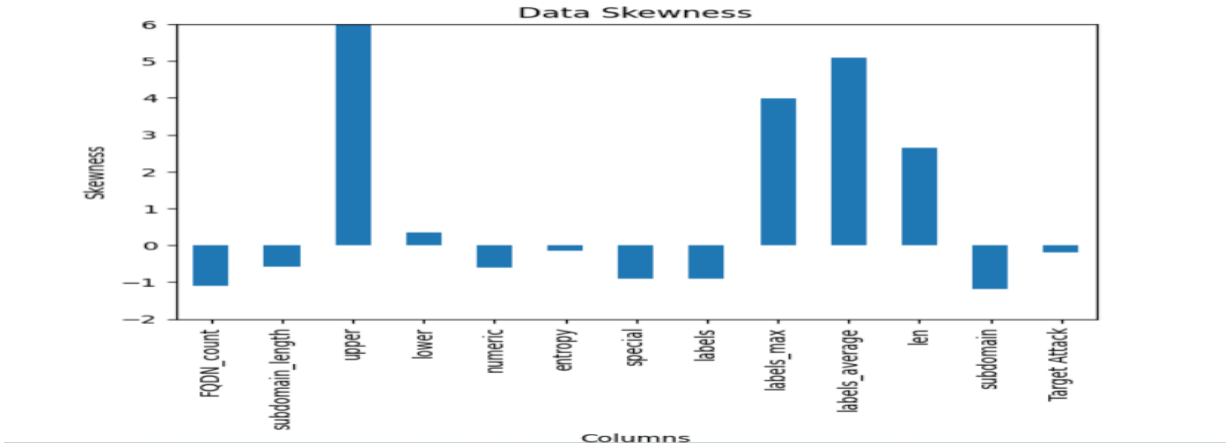
1. Data Analysis:

I have performed all the necessary parts in this section in the description. First, I checked each column's unique values and data types. The purpose of checking was to remove that column with only one unique value or unique values approximately equal to the records because these values do not affect the target variable. But all the unique values ranged from 2 to 35k, and the total records were approximately 250k. So, there was no need to delete the column at this point. But later, using other techniques, the number of features was reduced.

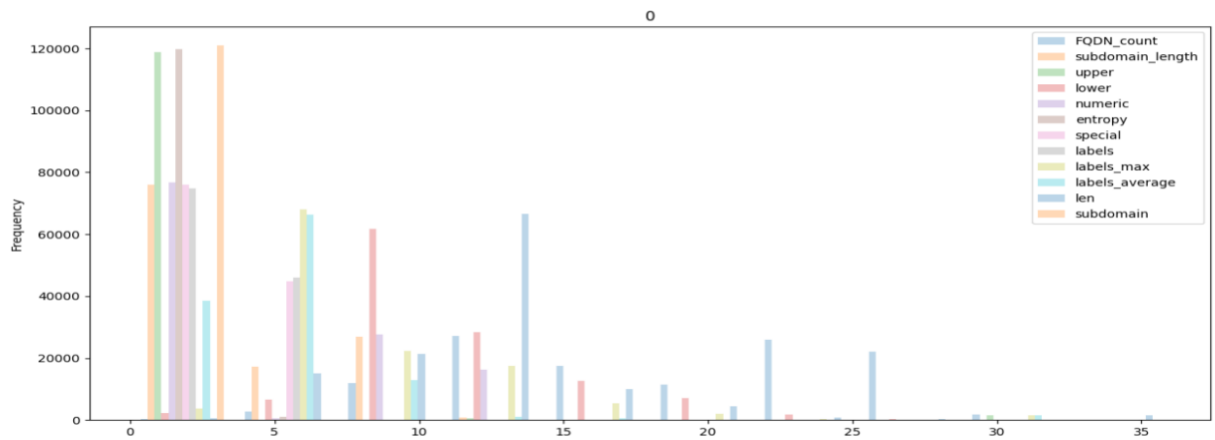
Then I check the number of classes in the target variable. There were only two classes. So, I check if either data is imbalanced or not. While analyzing, I found that "Class 0" has 120k records and "Class 1" has 147k records. So, data is biased towards "Class 1". Finally, I have shown the results using the plot. The problem with the imbalanced dataset is that it gives biased results towards the class with more records. We can handle it differently if it does not give good accuracy. But it gives 80 percent accuracy, which is considered good, so there was no need to remove biases.
Then I check the skewness of the data. For its checking, I used the built-in feature of skew in pandas. I checked it along the 0 axis, which represents the column-wise skewness. I also plotted the results. While showing the results, I found that some features were good while others were either highly positively skewed or negatively skewed.

Finally, I checked the data distribution. For this purpose, firstly, I used the built-in feature of describing pandas to check the mean, standard deviation, the minimum and maximum value of data. Then I check the distribution of each feature by target variable. Most features contain discrete values, but some contain continuous values, so it was important to check their distribution, but all the data was equally distributed.

```
plt.xlabel("Columns")
plt.ylabel("Skewness")
plt.title("Data Skewness")
plt.show()
```




```
# plotting data discription
data.plot.hist(by="Target Attack", figsize=(15, 15), alpha=0.3)
plt.show()
```



2. <u>Data Cleaning:</u>

In this section, firstly, I performed the null value check. It is important to handle null values because they can be problematic later while training models. Models cannot handle null values and give errors. While checking, I found only eight null values in one feature. It can be handled by removing rows with null values or filling them with multiple techniques since the number of rows with null values was only eight, which is nothing compared to the total records, which are 250k. So, I removed the rows with null values.

Then comes the categorical data conversion. It is also a basic and important task in data cleaning. Machine Learning models handle only numerical data, so I have to change all the categorical data into numerical using the encoding technique. For this purpose, I used the built-in features of pandas for encoding categorical data into numerical ones.

3. Data Selection:

While analyzing data, I discovered that the "time_stamp" feature does not affect data because it does not matter what time data comes. So, it was a useless feature. I removed it first. Then I used two methods for the selection of features. The first was wrapper methods. In this technique, I have used Pearson's correlation to check the correlation of each feature with the target variable. It tells how much a feature affects the target variable or how much the target variable is dependent on the other features. It gives values [-1 to 1]. The closer the value to 0, the greater it has to effect, so I chose the feature in the range of (-0.5 to 0.5). The second method was the recursive elimination of the method. Basically, It takes a model and a number of the best features we want and data. Then it makes the combination of features according to our number, in our case, 7, and returns features with more accuracy. This method is highly effective than the other one, but it also takes time. Then I selected those features which were returned by the Recursive Elimination method. Finally, I split the data into training and testing.

4. Model Training:

I chose the two models for this purpose. One was Gradient Boosting trees, and the other was Gaussian Naive Bayes. Then, I trained them using the training data.

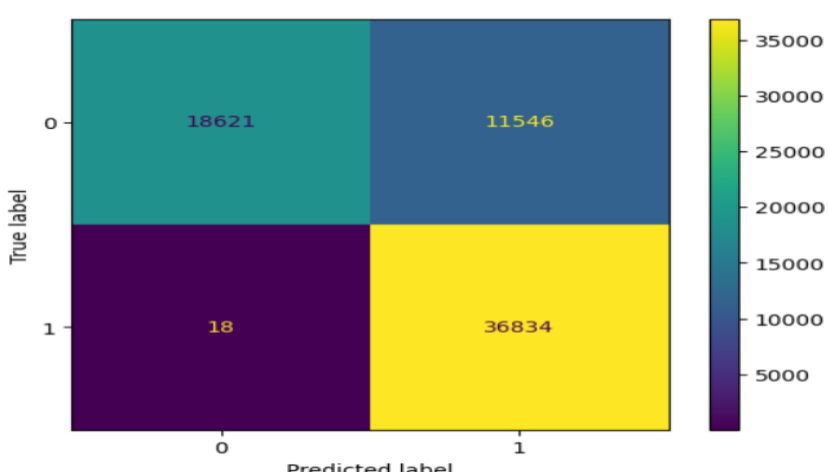
5. Model Evaluation:

Finally, for evaluation purposes, I checked the accuracy of models using the testing dataset. Also, I displayed the confusion matrix of data which tells the TP, TN, FP and FN of results.
So Gradient Boosting trees give better accuracy, so I saved this model for use in dynamic model evaluation.

1.XGBoost

```
# printing results of gradient boosting trees
xgb_prediction = xgb.predict(X_test)
print(f"Accuracy of XGBoost is {accuracy_score(xgb_prediction, y_test)}")
cm = confusion_matrix(y_test, xgb_prediction)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
disp.plot()
plt.show()
```

```
Accuracy of XGBoost is 0.8274519166206599
```

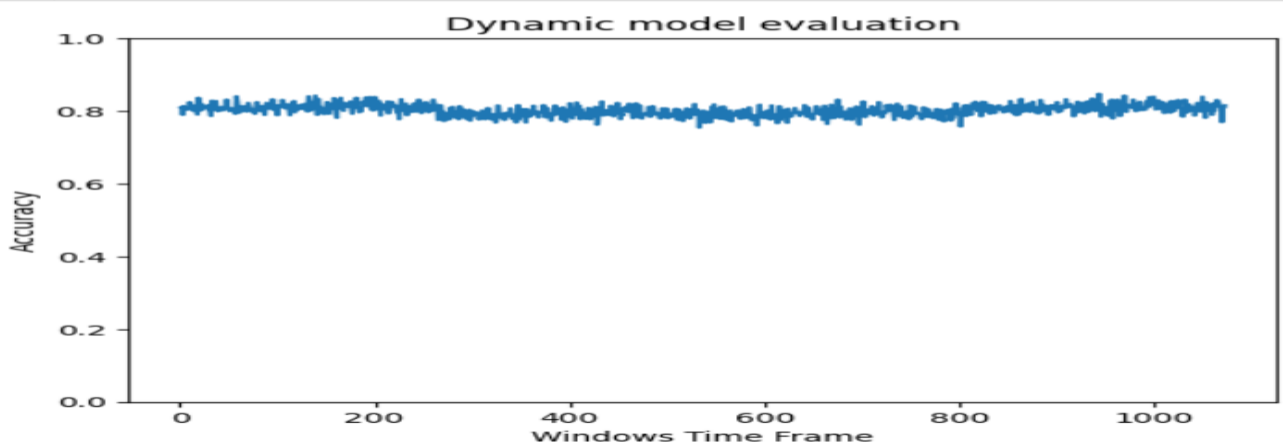


- **Dynamic Evaluation:**

I downloaded Docker, installed it, and ran it for this part which ran well. Then I ran the script that was given for this purpose. It took a few minutes to run because it downloaded some things. Then an image was created in Docker. After that, I run the producer’s code given by you. It took approximately 10 minutes. Finally, I ran the consumer script; it was successfully running.

Then I moved toward the dynamic code notebook. For this purpose, I loaded the Kafka dataset and deleted everything from the end until the data selection part. The selected data was saved as “kafka_clean_dataset.csv”. First, I run the producer’s code on this data. Then I ran the consumer code and made the windows of data in 10000 and checked the model accuracy and saved the results in an array. Then I checked the results, the results were good, and there was no need to train the model again. Because the static model’s accuracy was around 82 percent and the dynamic model gave accuracy greater than 80 percent. The accuracy was also plotted against each time frame.
It is better to retrain the model if it is not giving the accuracy you want, but it depends on the data. If the data is good and accuracy is still low, then the model should be trained but checking different parameters. Dynamic models also give accuracy equal to static in each window of size 10k. It is better to use the dynamic model in production when data continuously comes, and you can check accuracy in windows. Still, when you have all the data in the collective form, there is no need to check its accuracy in windows. We can use a static model with whole data to check accuracy.

```
# plotting results of each window
plt.plot(np.arange(1, len(results)+1), results)
plt.xlabel("Windows Time Frame")
plt.ylabel("Accuracy")
plt.ylim(0,1)
plt.title("Dynamic model evaluation")
plt.show()
```



Loading Models

```
xgb_static = joblib.load("static_model.pkl")
xgb_dynamic = joblib.load("static_model.pkl")
```